



دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



فاز دوم پروژه پایانی

استقرار Kubernetes و خودکارسازی CI/CD GitLab
تحلیل لاگ جام جهانی

مبانی رایانش ابری

نیمسال دوم ۱۴۰۴-۱۴۰۵

گزارش طراحی، پیاده‌سازی، خطایابی و آزمون

مشخصات گروه

سید حسام الدین حسینیان ۴۰۲۳۱۹۰۱

فروزان قویدل ۴۰۲۳۱۰۶۴

استاد درس: دکتر جوادی

تدریس‌یاران: مهرداد شیخ‌عباسی و محمدهادی ستاک

تابستان ۱۴۰۵

فهرست مطالب

فهرست مطالب

۳	۱ خلاصه اجرایی
۳	۲ خواندن دستورکار و تبدیل آن به برنامه اجرا
۴	۱.۲ وضعیت منابع در شروع
۵	۳ معماری نهایی سامانه
۶	۴ پیاده‌سازی زیرساخت Kubernetes
۶	۱.۴ ساخت کلاستر و namespace
۷	۲.۴ Storage و تفکیک داده
۷	۳.۴ RBAC و Secret ConfigMap
۷	۵ انتقال سرویس‌ها و Nginx
۷	۱.۵ سه backend
۸	۲.۵ Gateway و مسیر لاگ
۸	۶ تولید ترافیک به شکل Job Kubernetes
۹	۷ Registry و Runner GitLab داخل کلاستر
۹	۱.۷ CE GitLab
۹	۲.۷ Runner GitLab
۹	۳.۷ Registry و Kaniko
۹	۸ طراحی CI/CD GitLab
۱۰	۱.۸ Stage تست
۱۰	۲.۸ build Stage
۱۰	۳.۸ های Stage تا deploy تا publish
۱۰	۴.۸ اجرای نهایی پایپ‌لاین
۱۱	۹ Streaming Hadoop روی Kubernetes
۱۱	۱.۹ راه‌اندازی HDFS با مصرف کنترل‌شده
۱۱	۲.۹ Job ۱: Cleaning and Parsing
۱۲	۳.۹ Job ۲: Aggregation Nginx General
۱۲	۴.۹ Job ۳: Count Country-Entity
۱۲	۵.۹ Job ۴: Country by Entity Popular
۱۲	۶.۹ Job ۵: Summary Final
۱۲	۱۰ خروجی و تحلیل نتایج
۱۳	۱۱ Dashboard و انتشار نتیجه
۱۴	۱۲ کنترل CPU RAM و I/O
۱۴	۱۳ دفترچه خطاها و اصلاح مرحله‌به‌مرحله
۱۶	۱۴ بخش امتیازی روی Spark Kubernetes
۱۷	۱۵ آزمون‌ها و معیار پذیرش
۱۷	۱۶ شواهد نهایی قابل بررسی بدون دسترسی به لپ‌تاپ

۱۷	روش اجرای پروژه
۱۹	۱.۱۷ پیش‌نیاز
۱۹	۲.۱۷ راه‌اندازی GitLab و Runner
۱۹	۳.۱۷ کنترل وضعیت و آدرس‌ها
۱۹	۴.۱۷ اجرای Spark اختیاری
۱۹	۱۸ ساختار فایل‌های تحویلی
۲۰	۱۹ جمع‌بندی

خلاصه اجرایی

در فاز دوم، مسیر دستی فاز اول به یک سامانه واقعی روی Kubernetes منتقل شد. یک کلاستر kind با namespace ثابت cc-project ساخته شد. Registry Runner، GitLab CE، GitLab، HDFS، Nginx، backend، Job های ترافیک، MapReduce، اعتبارسنجی و تولید گزارش همگی داخل همان کلاستر اجرا شدند. کد پروژه در GitLab محلی push شد و Runner با Kubernetes executor همه مراحل را داخل pod های موقت اجرا کرد.

در اجرای نهایی، ۱۰۰،۰۰۰ درخواست فقط از Nginx عبور کرد. فایل log access شامل دقیقاً ۱۰۰،۰۰۰ شناسه یکتا بود و مجموع سه فایل backend نیز دقیقاً ۱۰۰،۰۰۰ رکورد شد. پنج Job واقعی Streaming Hadoop روی HDFS اجرا شدند؛ ۱۳ خروجی الزامی با validator کم حافظه تأیید شد؛ داشبورد HTML ساخته و با NodePort روی پورت ۹۰۰۰ منتشر شد.

وضعیت نهایی بخش اجباری

کلاستر، Registry، Runner، GitLab، پایپ لاین هشت مرحله ای، ترافیک ۱۰۰ هزار درخواست، پنج Job Hadoop، اعتبارسنجی خروجی و داشبورد قابل مشاهده همگی به صورت واقعی اجرا شده اند. اجرای نهایی پایپ لاین با شناسه ۱۶ و f507f6c3 commit در ۳۷۲ ثانیه با وضعیت موفق تمام شد. هیچ Secret یا token در مخزن و بسته تحویلی قرار نگرفته است.

خواندن دستورکار و تبدیل آن به برنامه اجرا

فایل ۲۵ صفحه ای فاز دوم ابتدا کامل خوانده شد. سپس الزام ها به ترتیب وابستگی مرتب شدند. نتیجه این بود که اجرای درست باید از زیرساخت شروع شود، بعد GitLab و Runner بالا بیایند، سپس build و deploy انجام شود و در پایان ترافیک، تحلیل و انتشار اجرا شوند. ترتیب کاری استفاده شده چنین بود:

۱. بررسی کد فاز اول، template گزارش و منابع میزبان؛
۲. ساخت کپی مستقل فاز دوم و نگه داشتن خروجی فاز اول بدون تغییر؛
۳. طراحی namespace، PVC، Service ها، Deployment ها و Job ها؛
۴. ساخت کلاستر kind و کنترل سلامت node؛
۵. راه اندازی Registry، GitLab CE و Runner داخل Kubernetes؛
۶. ایجاد پروژه، GitLab ثبت Runner و push مخزن؛
۷. طراحی پایپ لاین test تا publish و build بدون daemon Docker با Kaniko؛
۸. اجرای ترافیک واقعی و رفع اشکال مسیرهای PVC؛
۹. اجرای Streaming، Hadoop اعتبارسنجی ۱۳ خروجی و ساخت dashboard؛
۱۰. ثبت همه خطاهای واقعی، اندازه گیری منابع و آماده سازی بسته تحویل.

وضعیت منابع در شروع

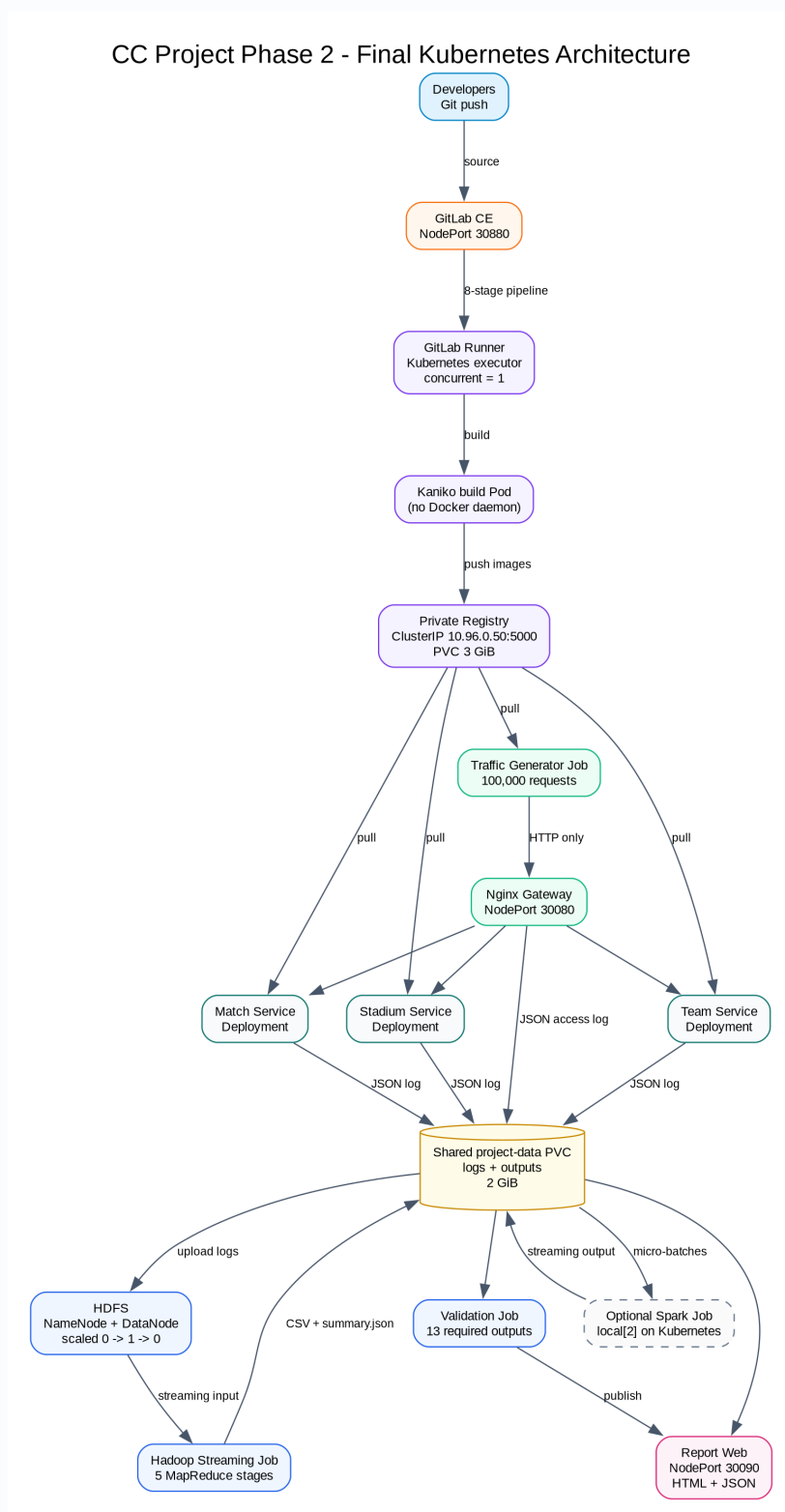
میزبان ۷/۴ گیگابایت RAM و ۱۶ پردازنده منطقی داشت. مقدار حافظه available در بخش‌های مختلف معمولاً بین ۱/۰ تا ۱/۸ گیگابایت بود. به همین علت از ابتدا سه تصمیم گرفته شد: Runner فقط یک Job همزمان اجرا کند، HDFS تنها در مرحله MapReduce روشن باشد و های‌بیلد سنگین به صورت ترتیبی انجام شوند.

جدول ۱: بودجه منابع بخش‌های پرمصرف

بخش	request	limit
CE GitLab	۱۵۰۰MiB / ۵۰۰m	۳۲۰۰MiB / CPU ۲
Runner GitLab	۶۴MiB / ۵۰m	۳۸۴MiB / ۵۰۰m
MapReduce	۳۸۴MiB / ۲۵۰m	۱۲۰۰MiB / CPU ۱
NameNode	۲۵۶MiB / ۱۰۰m	۸۰۰MiB / CPU ۱
DataNode	۲۲۴MiB / ۱۰۰m	۶۰۰MiB / ۶۰۰m
اختیاری Spark	۵۱۲MiB / ۲۵۰m	۱۶۰۰MiB / CPU ۲

ها request برای زمان‌بندی محافظه‌کارانه کوچک نگه داشته شدند، اما ها limit جلوی مصرف کنترل‌نشده را می‌گیرند. HDFS و Spark همزمان اجرا نمی‌شوند. هنگام Spark نیز GitLab و مسیر سرویس‌دهی موقتاً scale-down می‌شوند؛ بنابراین جمع های limit جدول به معنی رزرو همزمان حافظه نیست.

معماری نهایی سامانه



شکل ۱: گراف معماری نهایی فاز دوم

مسیر اصلی از چپ به راست در شکل دیده می‌شود. Developer کد را به GitLab داخلی می‌فرستد. GitLab یک pipeline هشت مرحله‌ای ایجاد می‌کند و Runner با Kubernetes executor برای هر Job یک pod موقت

می‌سازد. Kaniko imageها را بدون دسترسی به daemon Docker در Registry خصوصی می‌گذارد. De-ها image ployment همان commit را pull می‌کنند. Job ترافیک فقط با Service داخلی Nginx صحبت می‌کند. Nginx و ها JSON backend log را در PVC مشترک می‌نویسند. در مرحله تحلیل، NameNode و DataNode از صفر به یک scale می‌شوند، Hadoop Job خروجی را روی همان PVC قرار می‌دهد و پس از پایان دوباره به صفر scale می‌شوند. validator و dashboard آخرین مصرف‌کنندگان داده هستند.

جدول ۲: اجزای اصلی معماری

جزء	نقش	نوع Kubernetes
CE GitLab	نگهداری مخزن و مدیریت CI/CD	Ser- + Deployment PVC + vice
Runner GitLab	اجرای ها Job با Kubernetes executor	+ Deployment Secret + RBAC
Registry خصوصی	نگهداری های image هر commit و cache کانیکو	+ Deployment PVC + ClusterIP
سه backend	پاسخ‌گویی و نوشتن log service	سه Deployment و سه Service
Gateway Nginx	تنها ورودی ترافیک و تولید log access	+ Deployment NodePort
project-data HDFS	لاگ‌ها، خروجی Hadoop و dashboard ذخیره ورودی و خروجی Streaming	PVC دو گیگابایتی دو StatefulSet و دو Service
MapReduce Web Report	اجرای پنج مرحله تحلیل انتشار HTML و JSON	Job + Deployment NodePort

پیاده‌سازی زیرساخت Kubernetes

ساخت کلاستر و namespace

فایل k8s/kind-config.yaml پورتهای NodePort لازم را از node به میزبان منتقل می‌کند. namespace پروژه cc-project است. اسکریپت scripts/phase2/create_cluster.sh idempotent نوشته شد: اگر کلاستر وجود داشته باشد آن را پاک نمی‌کند، context درست را انتخاب می‌کند، تا Ready شدن node منتظر می‌ماند و سپس storage را apply می‌کند.

containerd در namespace شبکه node اجرا می‌شود و DNS داخلی ها pod را ندارد. برای اینکه نام registry.cc-project.svc هنگام pull image نیز resolve شود، اسکریپت ساخت کلاستر نگاشت ثابت 10.96.0.50 را داخل hosts نود ثبت می‌کند. این مورد پس از مشاهده توقف pull image به اسکریپت اضافه شد.

Storage و تفکیک داده

چهار PVC مستقل تعریف شد:

- project-data با ظرفیت ۲Gi برای لاگ و خروجی تحلیل؛
- registry-data با ظرفیت ۳Gi برای image؛ cache؛
- gitlab-data با ظرفیت ۸Gi برای repository و پایگاه داده؛
- gitlab-config با ظرفیت ۱Gi برای کلیدها و تنظیمات. GitLab.

جداکردن /etc/gitlab/data از یک اصلاح مهم بود. در نسخه اولیه config روی emptyDir قرار داشت. پس از جایگزینی pod کلیدهای encryption دوباره ساخته شدند و GitLab هنگام خواندن های token قبلی خطای OpenSSL::Cipher داد. داده GitLab یک بار در محیط آزمایش بازسازی و پس از آن config روی PVC پایدار قرار گرفت.

Secret ConfigMap و RBAC

پیگیری Nginx و Web Report در ConfigMap قرار گرفت. گذرواژه اولیه، token authentication GitLab، مربوط به Runner و token API کوتاه‌عمر در Secret یا فایل محلی با mode برابر ۰۶۰۰ نگهداری شدند و مسیر phase2-secrets در gitignore قرار گرفت.

ServiceAccount مربوط به Runner فقط مجوزهای لازم برای pod، Job، State- Deployment، getnamespace/ ConfigMap Service، fulSet، و PVC در namespace پروژه را دارد. برای فرمان cc-project یک ClusterRole محدود با resourceNames تعریف شد. آزمون RBAC نشان داد get روی همان namespace مجاز و list همه namespaces مجاز است.

انتقال سرویس‌ها و Nginx

سه backend

سرویس‌های team match و stadium از کد فاز اول استفاده می‌کنند، اما اکنون هر کدام Deployment و Service ClusterIP مستقل دارند. برای هر request container و limit مشخص شد. های probe readiness و liveness روی /health هستند.

فایل‌های لاگ باید در این مسیرهای مشترک نوشته شوند:

```
/project-data/data/service_logs/match_service.log
/project-data/data/service_logs/team_service.log
/project-data/data/service_logs/stadium_service.log
```

برنامه‌ها متغیر SERVICE_LOG_PATH را می‌خوانند. در یک اجرای واقعی مشخص شد manifest اشتباهاً متغیر LOG_PATH می‌فرستاد؛ به همین دلیل حدود ۲۹MB لاگ داخل filesystem موقت سه container

باقی مانده بود. نام متغیر و مسیر هر سه Deployment اصلاح شد و یک check regression به تست‌های CI اضافه شد.

Gateway و مسیر لاگ

Nginx تنها درگاه عمومی API است. های route `/api/stadiums` و `/api/teams`، `/api/matches` به Service داخلی متناظر proxy می‌شوند. های request header، ID کشور و scenario بدون تغییر به backend می‌رسند. check health از log access خارج شده است تا داده تحلیل فقط شامل درخواست‌های پروژه باشد.

log access با فرمت JSON و buffer محدود در مسیر زیر ذخیره می‌شود:

```
/project-data/data/nginx/nginx_access.log
```

در اجرای اول فاز دو، Nginx فایل را در `/project-data/nginx/nginx_access.log` می‌نوشت ولی Hadoop مسیر `data/nginx` را می‌خواند. همان اجرا پیش از MapReduce متوقف شد، فایل اشتباه پاک شد، ConfigMap و initContainer اصلاح شدند و ترافیک از ابتدا تکرار شد.

تولید ترافیک به شکل Job Kubernetes

مولد ترافیک یک Job با ۳۲ worker محدود است. مقصد آن فقط `http://nginx-gateway:80` است و هیچ آدرس backend در container وجود ندارد. seed ثابت ۱۴۰۵ باعث می‌شود توزیع درخواست‌ها تکرارپذیر باشد. پنج scenario عادی، popular، error server invalid و slow تولید می‌شوند.

پیش از هر اجرا، stage ترافیک این ترتیب را انجام می‌دهد:

۱. های Nginx replica و سه backend را صفر می‌کند؛
۲. تا حذف pod قدیمی Nginx منتظر می‌ماند تا buffer قبلی باز نماند؛
۳. Job پاک‌سازی PVC را اجرا می‌کند و فایل‌ها را صفر می‌کند؛
۴. ها Deployment را دوباره یک می‌کند و rollout هر چهار مورد را کنترل می‌کند؛
۵. Job ترافیک را با tag image همان commit می‌سازد؛
۶. تا condition برابر Complete منتظر می‌ماند و log خلاصه را چاپ می‌کند.

شمارش اجرای مبنا

فایل Nginx دقیقاً ۱۰۰،۰۰۰ خط و ۱۰۰،۰۰۰ request ID یکتا داشت. فایل‌های stadium و team match، log access به ترتیب ۲۸،۱۶۲، ۴۹،۶۱۹ و ۲۲،۲۱۹ خط داشتند؛ جمع آن‌ها نیز دقیقاً ۱۰۰،۰۰۰ است. حجم log access حدود ۲۸،۷MiB و مجموع service logها حدود ۲۷،۴MiB بود.

برای خطاهای گذرای شبکه، Job ۱ رکوردها را با کلید ترکیبی source و request ID مرتب و duplicate احتمالی را حذف می‌کند. یک تست جداگانه دو نسخه از یک درخواست را به mapper/reducer می‌دهد و خروجی یک رکورد را کنترل می‌کند.

Registry و Runner GitLab، داخل کلاستر

CE GitLab

GitLab CE نسخه 17.11.7-ce.0 داخل Deployment اجرا شد. قابلیت‌های غیرضروری مانند monitoring داخلی برای کاهش مصرف خاموش شدند. probe readiness از curl داخلی GitLab روی 127.0.0.1/127.0.0.1/readiness - استفاده می‌کند. دسترسی میزبان از http://localhost:8929 فراهم است، ولی Runner و سرویس‌های داخل کلاستر از DNS داخلی gitlab.cc-project.svc استفاده می‌کنند. اولین GitLab image pull حدود ۶ دقیقه و ۸ ثانیه طول کشید و image فشرده حدود ۱/۷۸GB بود. هنگام migration، مصرف node حدود ۲/۰GiB و CPU به‌طور لحظه‌ای نزدیک ۱۰۰ درصد یک هسته شد. پس از پایدار شدن، GitLab بخش اصلی حافظه را به Sidekiq Puma و PostgreSQL اختصاص داد.

Runner GitLab

Runner نسخه 17.11.1 در همان namespace ثبت شد. executor برابر kubernetes و concurrent برابر ۱ است. برای هر Job سه container معمول Runner یعنی helper build و init-permissions در یک pod موقت ایجاد می‌شود. image helper رسمی از Docker Hub استفاده شد، چون CDN مربوط به registry.gitlab در شبکه آزمایش پاسخ ۴۰۳ می‌داد.

Kaniko و Registry

Registry نسخه ۲/۸/۳ با ClusterIP ثابت 10.96.0.50:5000 و PVC سه گیگابایتی اجرا شد. im-های age هر commit در مسیر <short-sha>:cc-phase2/<service> ذخیره می‌شوند. Kaniko جایگزین Docker-in-Docker شد تا Runner به socket میزبان دسترسی نداشته باشد. image اصلی Kaniko روی gcr در این شبکه ۴۰۳ می‌داد؛ chinayin/kaniko-project:mirror v1.23.2-debug استفاده شد. Registry داخلی HTTP است، بنابراین های insecure flag -- و insecure-registry به executor داده شدند.

طراحی CI/CD GitLab

فایل gitlab-ci.yml شامل هشت stage است:

test → build → deploy → traffic → mapreduce → validate → report → publish

Stage تست

در stage اول این موارد اجرا می‌شوند:

- compile نحوی همه فایل‌های Python؛
- bash - n برای همه اسکریپت‌های shell؛
- اجرای محلی همان پنج mapper/reducer روی fixture کوچک؛
- تست request duplicate در Job ۱؛
- کنترل نام متغیر و مسیر PVC سه backend؛
- اعتبارسنجی ۱۳ خروجی موجود.

build Stage

برای جلوگیری از تداخل filesystem، Kaniko هر image یک Job مستقل دارد. به دلیل concurrent برابر یک، هاب Job به ترتیب اجرا می‌شوند. team، traffic stadium و phase۲-tools از image داخلی match همان commit به عنوان base استفاده می‌کنند تا وابستگی به CDN خارجی کم شود.

Hadoop image در ابتدا از یک base قدیمی دو گیگابایتی ساخته می‌شد و snapshot آن I/O زیادی ایجاد کرد. نسخه نهایی از eclipse-temurin:11-jre-jammy و Python و Hadoop ۳.۴.۳ ساخته شد. دانلود Hadoop با aria۲ و هشت connection انجام شد؛ فایل ۴۹۱MB در حدود ۴ دقیقه و ۱۲ ثانیه دریافت شد. برای build این image، Runner limit memory برابر ۲۵۰۰Mi و caching compressed غیرفعال شد.

های Stage deploy تا publish

deploy ها manifest را apply می‌کند و dev tag را با commit SHA عوض می‌کند. traffic داده قبلی را پاک و ۱۰۰ هزار درخواست تولید می‌کند. HDFS mapreduce را scale-up و بعد از پایان scale-down می‌کند. validate همه schema و عدد ۱۰۰,۰۰۰ را کنترل می‌کند. report داشبورد را با کاربر non-root می‌سازد و های artifact، JSON HTML و CSV را به GitLab می‌دهد. Deployment publish وب را restart و دریافت summary . json از Service داخلی را آزمایش می‌کند.

اجرای نهایی پایپ‌لاین

پس از اعمال آخرین اصلاح health، commit check، push f507f6c3 شد. GitLab برای آن پایپ‌لاین شماره ۱۶ را ساخت. صف اولیه ۱۵ ثانیه و زمان اجرای ثبت‌شده ۳۷۲ ثانیه بود. زمان پایان در GitLab برابر ۲۵ ژوئیه ۲۰۲۶، ساعت ۰۱:۱۸ به وقت تهران ثبت شد.

جدول ۳: نتیجه های Job پایپ لاین نهایی شماره ۱۶

مدت تقریبی	وضعیت	Job یا گروه هاJob
۶ ثانیه	موفق	test-code
۱۷ ثانیه	موفق	build-match-service
۶ تا ۸ ثانیه برای هرکدام	هر چهار موفق	build-team / stadium / traffic / tools
۲۸ ثانیه	موفق	build-hadoop-streaming
۱۱ ثانیه	موفق	deploy-kubernetes
۱۱۷ ثانیه	موفق	generate-traffic
۱۳۸ ثانیه	موفق	run-mapreduce
۹ ثانیه	موفق	validate-output
۹ ثانیه	موفق	generate-report
۹ ثانیه	موفق	publish-report

Kubernetes روی Streaming Hadoop

راه اندازی HDFS با مصرف کنترل شده

NameNode و DataNode به صورت StatefulSet تعریف شدند، اما replica پیش فرض آن‌ها صفر است. تنها Job run-mapreduce آن‌ها را یک می‌کند. های HDFS image روی میزبان موجود بودند؛ به جای دانلود دوباره دو image حدود ۲/۰۶GB با یک pipe مستقیم ctrimagesimport dockersave وارد containerd شدند. import هر دو image در مجموع ۱۶/۴ ثانیه طول کشید و فایل موقت چندگیگابایتی ساخته نشد.

پس از NameNode port restart ممکن است Ready باشد ولی HDFS هنوز در Mode Safe باشد. در یکی از اجراها upload با پیام Cannot create file; Name node is in safe mode متوقف شد. اسکرینیت نهایی تا ۱۸۰ ثانیه، هر دو ثانیه وضعیت hdfsdfsadmin-safemodeget را می‌خواند و فقط پس از OFF شدن نوشتن را شروع می‌کند.

Job ۱: Cleaning and Parsing

چهار فایل خام به HDFS فرستاده می‌شوند. JSON mapper را parse می‌کند، فیلدهای اجباری و بازه code status و زمان را می‌سنجد و رکورد را به service nginx یا tag invalid می‌دهد. reducer با کلید hash شده request ID، های retry تکراری را حذف می‌کند. خروجی‌های این مرحله دو CSV تمیز و فایل invalid هستند.

Job ۲: Aggregation Nginx General

برای endpoint service و scenario تعداد کل، موفق، خطای ۴xx، خطای ۵xx، نرخ خطا و میانگین re-time sponse محاسبه می‌شود. همین خروجی منبع summary نهایی است.

Job ۳: Count Country-Entity

log service تمیز به سه دسته، day match team و stadium/city نگاشت می‌شود و تعداد درخواست هر موجودیت برای هر کشور جمع می‌شود.

Job ۴: Country by Entity Popular

خروجی Job ۳ دوباره پردازش می‌شود. برای هر کشور محبوب‌ترین تیم، روز بازی و ورزشگاه انتخاب می‌شود. در تساوی، مرتب‌سازی نام نتیجه را deterministic می‌کند.

Job ۵: Summary Final

Job آخر خروجی‌های Job ۲ تا ۴ را با metadata پیش‌بینی ترکیب می‌کند و outputs/final/summary.json را می‌سازد. پس از آن validator script ساختار همه خروجی‌ها را کنترل می‌کند.

✓ اجرای واقعی پنج Job

در پایپ‌لاین نهایی، MapReduce Job از شروع scale-up تا پایان تحلیل ۱۳۸ ثانیه طول کشید. pod پردازش هیچ restart یا OOM نداشت. هر پنج marker اجرایی، اعتبارسنجی ۱۳ فایل و پیام MapReduce pipeline completed successfully در trace ثبت شد.

📖 خروجی و تحلیل نتایج

جدول ۴: آمار سرویس‌ها در اجرای ۱۰۰ هزار درخواست

service	کل	موفق	۴xx	۵xx	میانگین ms
match-service	۲۸,۱۶۲	۲۷,۲۹۲	۵۶۳	۳۰۷	۳۲,۳۱۱
stadium-service	۲۲,۲۱۹	۲۱,۵۶۵	۴۵۱	۲۰۳	۳۲,۸۳۵
team-service	۴۹,۶۱۹	۴۸,۱۵۸	۹۷۰	۴۹۱	۲۶,۰۵۳

team-service با ۴۹,۶۱۹ درخواست پرتقاضاترین سرویس بود. نرخ خطای match-service برابر ۳/۰۸۹۳ درصد و از دو سرویس دیگر کمی بیشتر بود. کندترین endpoint طبق summary، /api/stadiums / شد.

جدول ۵: آمار ها scenario

scenario	تعداد	موفق	نرخ خطا	میانگین ms
normal	۹۱,۴۶۶	۹۱,۴۶۶	۰%	۲۵/۶۸۴
popular	۵,۰۷۲	۵,۰۷۲	۰%	۲۵/۳۰۷
invalid	۱,۹۸۴	۰	۱۰۰%	۲۵/۵۰۷
server_error	۱,۰۰۱	۰	۱۰۰%	۲۵/۶۵۲
slow	۴۷۷	۴۷۷	۰%	۷۹۳/۳۰۰

سناریوی slow فقط ۴۷۷/ درصد ترافیک بود، ولی میانگین زمان آن حدود ۷۹۳ms شد؛ پس میانگین کلی ها endpoint را بدون ایجاد load کنترل نشده افزایش داد.

summary نهایی این نتایج را ثبت کرد:

- محبوب ترین تیم کلی: آرژانتین؛
- پرتقاضاترین روز بازی: ۲۵ ژوئن ۲۰۲۶؛
- پرتقاضاترین ورزشگاه: Stadium: Jersey New York New
- قهرمان پیش بینی شده: آرژانتین؛
- فینال پیش بینی شده: فرانسه مقابل آرژانتین؛
- محل فینال پیش بینی شده: Stadium. Jersey New York New

Dashboard و انتشار نتیجه

اسکرپت `summary report-web/generate_dashboard.py` و `stats service` را می خواند و HTML مستقل تولید می کند. `image` ابزار با `UID` برابر ۱۰۰۰۱ اجرا می شود. در اولین اجرای `report` پوشه `outputs/final` توسط `Hadoop` با مالک `root` و `mode` برابر ۰۷۵۵ ساخته شده بود و نوشتن `index` با `PermissionError` متوقف شد.

نسخه نهایی یک `initContainer` بسیار کوچک دارد که فقط `group` همان پوشه را ۲۰۰۰ و `mode` را ۰۷۷۵ می کند. `container` اصلی همچنان `non-root` باقی می ماند. `test smoke` موفق پیام زیر را ثبت کرد:

```
Dashboard written to /project-data/outputs/final/index.html
<title>World Cup Log Analytics
```

`Web Report` فایل های HTML و JSON را `mount read-only` می کند. دسترسی میزبان از `http://localhost:9000/index.html` و دسترسی داخلی از `report-web Service` است.

کنترل، CPU RAM، I/O

جدول ۶: اقدام‌های انجام‌شده برای کنترل منابع

بخش	اقدام
Runner	concurrent برابر ۱؛ های build مستقل و ترتیبی
GitLab	خاموش کردن monitoring غیرضروری و تعیین request/limit
Kaniko	base Registry، cache داخلی و caching compressed خاموش فقط برای Hadoop
Traffic	۳۲ worker محدود و سقف ۲ CPU و ۱۹۲MiB
HDFS	replica صفر در حالت عادی و scale موقت در MapReduce
Validator	خواندن CSV به صورت streaming به جای list کامل
Image انتقالی	pipe مستقیم به containerd و عدم ساخت archive موقت
Spark	توقف GitLab و ها backend هنگام اجرای بخش اختیاری

کمترین حافظه available مشاهده شده در اجرای Hadoop حدود ۱/۰ تا ۱/۲GiB بود. هیچ pod پردازش restart نشد. نسخه اول validator با limit برابر ۱۲۸MiB OOMKilled شد، چون دو CSV بزرگ را کامل در list می ریخت. نسخه streaming روی خروجی ۱۰هزار رکورد با tracemalloc حدود ۰/۹MiB peak تخصیص Python داشت و همان ۱۳ بررسی schema را حفظ کرد.

دفترچه خطاها و اصلاح مرحله به مرحله

خطاهای زیر در اجرای واقعی دیده شدند. این موارد حذف یا پنهان نشده اند، چون بخش مهم فاز دوم خطایابی محیط واقعی Kubernetes و CI/CD است.

۱. **Load مستقیم kind:** image چندسکوپی در Docker ۲۹ و containerd ۲ digest ناقص داد. برای های image معمول pull مستقیم و برای های image HDFS import مستقیم با ctr استفاده شد.
۲. **GitLab: Readiness** آدرس loopback و مسیر curl اولیه درست نبود. probe به curl داخلی GitLab و endpoint رسمی readiness تغییر کرد.
۳. **helper: Runner** دانلود از CDN مربوط به registry.gitlab پاسخ ۴۰۳ داشت. helper رسمی همان نسخه از Hub Docker انتخاب شد.
۴. **کلیدهای GitLab:** config موقت باعث تغییر کلید encryption پس از تعویض pod شد. PVC جداگانه gitlab-config اضافه شد.
۵. **Image ابزار CI:** tag انتخابی bitnami/kubectl وجود نداشت. image alpine/k8s:1.32.3 جایگزین شد.

۶. **Kaniko روی gcr:** دسترسی ۴۰۳ بود؛ Hub Docker mirror استفاده شد.
۷. **Build اولیه Hadoop:** base قدیمی دو گیگابایتی I/O بسیار زیاد ایجاد کرد و یک بار EOF و یک بار OOM رخ داد. base سبک Java ۱۱ و Hadoop ۳.۴.۳ ساخته شد و limit همان Job به ۲۵۰۰MiB رسید.
۸. **دانلود Hadoop:** curl تک connection سرعت حدود ۰/۳MB/s داشت. aria۲ با هشت con- nection میانگین حدود ۱/۹MiB/s داد.
۹. **namespace: RBAC** Runner مجوز get namespace نداشت. ClusterRole فقط برای نام cc-project افزوده شد و list همچنان غیرمجاز ماند.
۱۰. **تداخل:ها:build** اجرای شش Kaniko پشت سر هم در یک pod باعث شد بعضی هاimage فایل های base را درست نداشته باشند. هر image به Job مستقل تبدیل شد.
۱۱. **دسترسی HTTP: Registry** pull داخلی ابتدا HTTPS فرض شد. insecure flag و تنظیم mirror اضافه شد.
۱۲. **Registry DNS در node:** containerd به pod DNS دسترسی نداشت. IP ثابت Service در node hosts و اسکریپت ایجاد کلاستر ثبت شد.
۱۳. **مسیر log: access** Nginx در شاخه ای متفاوت از ورودی Hadoop می نوشت. MapReduce متوقف، مسیر اصلاح، فایل اشتباه حذف و ترافیک از نو تولید شد.
۱۴. **چهار retry شبکه ای:** در یک اجرای آزمایشی ۱۰۰،۰۰۴ خط خام و چهار request ID تکراری دیده شد. dedup مبتنی بر request ID در Job ۱ و تست رگرسیون اضافه شد.
۱۵. **متغیر لاگ: backend** LOG_PATH توسط برنامه خوانده نمی شد. SERVICE_LOG_PATH و مسیر کامل PVC برای هر سه سرویس اصلاح شد.
۱۶. **OOM: Validator** list کردن های CSV بزرگ بیش از ۱۲۸MiB مصرف کرد. validator سطر به سطر بازنویسی شد.
۱۷. **Mode: Safe HDFS** readiness شبکه پیش از امکان نوشتن برقرار شد. انتظار صریح حداکثر ۱۸۰ ثانیه برای Mode Safe OFF اضافه شد.
۱۸. **ثبت IP جدید: DataNode** پس از ساخت دوباره pod، NameNode به علت نبود reverse-DNS، IP جدید DataNode را رد کرد. check hostname مخصوص محیط container غیرفعال شد؛ احراز هویت و شبکه namespace تغییری نکرد.
۱۹. **Permission داشبورد:** Hadoop پوشه final را root:root ساخته بود. initContainer فقط group و permission write همان پوشه را آماده کرد.
۲۰. **اتصال هنگام rollout گزارش:** اولین wget به endpoint در حال حذف گیر کرد، در حالی که درخواست بعدی پاسخ ۲۰۰ داشت. برای check health انتشار، timeout پنج ثانیه ای و retry محدود به فرمان wget اضافه شد.

۲۱. **خاموش شدن لپ تاپ میان اجرا:** اجرای شماره ۱۴ هنگام تولید ترافیک با قطع برق میزبان نیمه کاره ماند. پس از روشن شدن دوباره، کلاستر، kind های PVC، Registry GitLab و داده پروژه بررسی شدند و سالم بودند. Job نیمه کاره مبنای نتیجه قرار نگرفت؛ pipeline تازه از آخرین commit اجرا شد و اجرای شماره ۱۶ از test تا publish کامل و موفق پایان یافت.

۲۲. **تفکیک شواهد دو فاز:** در بازبینی نهایی پیش از انتشار GitHub مشخص شد پوشه evidence فاز دوم هنوز تصویرها و متن های اجرای Compose Docker فاز اول را نگه داشته است. آن فایل ها برای اثبات Kubernetes مناسب نبودند و از فاز دوم کنار گذاشته شدند؛ نسخه اصلی آن ها در گزارش فاز اول محفوظ ماند. پوشه evidence فاز دوم با وضعیت واقعی، namespace، اطلاعات Pipeline ۱۶، شمارش لاگ ها، خروجی، publish MapReduce و Spark روی Kubernetes از نو ساخته شد.

بخش امتیازی Spark روی Kubernetes

طراحی بخش اختیاری

کد manifest اجرای Spark و Job خروجی دادن ها batch در پوشه های spark/ و k8s/spark/ قرار دارند. ورودی و خروجی هر دو از PVC مشترک استفاده می کنند و اجرای Spark مستقل از مسیر اجباری GitLab/Hadoop است.

اسکرپت run_spark_kubernetes.sh برای کنترل منابع ابتدا، GitLab، Nginx Runner و back-ها را scale-down می کند. ConfigMap برنامه Spark را می سازد، Spark Job را اجرا می کند، های batch اتمیک را از لاگ های واقعی صادر می کند و پس از complete شدن هر دو، Job log نتیجه را نمایش می دهد.

نتیجه اجرای واقعی Spark

image رسمی apache/spark:3.5.5 بدون فایل موقت و با pipe مستقیم وارد containerd شد. Job با [2]local، حافظه driver برابر ۷۶۸MiB و limit کل ۱۶۰۰MiB اجرا شد و در ۶۸ ثانیه، بدون restart، کامل شد. صادرکننده پنج batch دویست ستری Nginx و پنج batch دویست ستری service ساخت. Streaming Structured در پنجره ۱۰ ثانیه ای ۴۹۴ درخواست، team ۲۸۰ درخواست match و ۲۲۶ درخواست stadium را در اولین batch دید و تیم محبوب را برای هشت کشور محاسبه کرد. برای نمونه، آرژانتین با ۱۱۳ مشاهده برای کشور آرژانتین و ژاپن با ۸۱ مشاهده برای کشور ژاپن در خروجی زنده ثبت شدند. پس از پایان، trap اسکرپت، GitLab، Nginx Runner و هر سه backend را بازیابی کرد و rollout همه آن ها موفق بود.

آزمون ها و معیار پذیرش

جدول ۷: ماتریس آزمون فاز دوم

آزمون	نتیجه	شاهد
سلامت node و namespace	موفق	node برابر Ready و namespace برابر cc-project
GitLab داخل Kubernetes	موفق	Ready Deployment و صفحه ورود روی ۸۹۲۹
executor Kubernetes Runner	موفق	pod موقت برای هر Job و online Runner
build با Kaniko	موفق	شش image با commit tag در Registry خصوصی
deploy سرویس‌ها	موفق	rollout چهار Deployment اصلی
ترافیک فقط از Nginx	موفق	مقصد Job فقط nginx-gateway و ۱۰۰هزار ID یکتا
لاگ مشترک	موفق	۱۰۰هزار Nginx و ۱۰۰هزار log service روی PVC
پنج Hadoop Job	موفق	پنج marker و پیام completion
اعتبارسنجی	موفق	۱۳ فایل و total برابر ۱۰۰,۰۰۰
Dashboard	موفق	JSON HTML، و NodePort ۹۰۰۰
محدودیت منابع	موفق	صفر restart در Hadoop و validator با حافظه ثابت
Streaming Structured Spark	موفق	ده batch، پنجره زنده، زمان ۶۸ ثانیه و صفر restart

📖 شواهد نهایی قابل بررسی بدون دسترسی به لپ‌تاپ

GitLab این پروژه داخل کلاستر محلی اجرا شده است و پس از تحویل، مصحح نمی‌تواند آدرس localhost:8929 لپ‌تاپ را باز کند. به همین دلیل فقط به تصویر صفحه GitLab اکتفا نشد. وضعیت نهایی با API خود GitLab و فرمان‌های Kubernetes خوانده، از ها Secret پاک و در پوشه report/evidence ذخیره شد. تمام فایل‌ها متنی هستند تا در GitHub مستقیماً دیده و مقایسه شوند.

جدول ۸: فهرست شواهد واقعی فاز دوم

فایل	محتوای قابل بررسی
01_kubernetes_status.txt	Deploymentها، StatefulSetها و Serviceهای namespace PVCها،
02_gitlab_pipeline_16.txt	commit زمان کل و وضعیت و مدت هر Job ۱۳
03_traffic_and_outputs.txt	شمارش لاگ، شناسه یکتا و summary نهایی
04_mapreduce_and_validation.txt	هفت گام اجرا، پنج Job تحلیلی و اعتبارسنجی ۱۳ فایل
05_dashboard_and_publish.txt	rollout وب، پاسخ‌های HTTP ۲۰۰ و نتیجه publish
06_spark_kubernetes.txt	وضعیت Job، پنجره زنده، limit حافظه و بازیابی سرویس‌ها
07_security_and_reproduction.txt	کنترل Secret و مسیر بازتولید کامل سامانه

```

Pipeline 16 | commit f507f6c3 | status success | duration 372 s
traffic-generator:
  Completed: 100,000 in 98.30s (1017.3 req/s)
data checks:
  nginx lines=1000000, unique request IDs=1000000
  backend lines=28162+49619+22219=1000000
mapreduce:
  five analysis jobs completed
  Validated 13 required output files.
publish:
  dashboard HTTP=200, summary HTTP=200
spark:
  status=Complete, duration=68s, restarts=0
    
```

قابلیت متمیزی مخزن GitHub

کد، ها،manifest اسکریپت نصب، داده واقعی، خروجی‌های تحلیل، گزارش PDF و شواهد اجرای نهایی در یک commit قرار گرفته‌اند. بزرگ‌ترین فایل ۲۸/۶MiB است، بنابراین هیچ فایل از محدودیت انتشار معمول GitHub عبور نمی‌کند. مصحح می‌تواند بدون دسترسی به هاSecret نتیجه را بررسی کند و در صورت نیاز کل محیط را از روی README بازتولید کند.

روش اجرای پروژه

پیش‌نیاز

ابزارهای Docker، kind، kubectl، git، curl و openssl لازم هستند. کلاستر با این فرمان ساخته می‌شود:

```
bash scripts/phase2/create_cluster.sh
bash scripts/phase2/load_core_images.sh
```

راه‌اندازی GitLab و Runner

```
bash scripts/phase2/deploy_gitlab.sh
bash scripts/phase2/configure_gitlab.sh
bash scripts/phase2/wait_for_pipeline.sh
```

اسکرپت token configure کوتاه‌عمر می‌سازد، پروژه را ایجاد می‌کند، Runner را ثبت می‌کند و مخزن را push می‌کند. token در خروجی چاپ یا commit نمی‌شود.

کنترل وضعیت و آدرس‌ها

```
bash scripts/phase2/collect_status.sh
kubectl -n cc-project get pods,svc,jobs,pvc
```

http://localhost:8929 GitLab: ●

http://localhost:8080 API: Nginx ●

http://localhost:9000/index.html Dashboard: ●

اجرای Spark اختیاری

```
IMAGE_TAG=<successful-short-sha> \
bash scripts/phase2/run_spark_kubernetes.sh
```

ساختار فایل‌های تحویلی

```
code/
.gitlab-ci.yml
k8s/
  base/      # namespace, PVC, Registry, backend, Nginx, HDFS, report
  jobs/      # traffic, reset, MapReduce, validator, report
  gitlab/    # GitLab CE
  runner/    # RBAC and Runner
```

```

    spark/          # optional Spark workload
match-service/
team-service/
stadium-service/
traffic-generator/
mapreduce/         # five mapper/reducer pairs
hadoop-k8s/
phase2-tools/
report-web/
scripts/phase2/
tests/
docs/architecture.{dot,svg,png}
report/
  report.tex
  report.pdf
  architecture.png
evidence/          # sanitized Kubernetes, GitLab CI, Hadoop and Spark proof
    
```

نام بسته نهایی مطابق دستورکار به صورت زیر است:

CC_Project_Phase2_40231901_40231064.zip

جمع بندی

در این فاز فقط manifest نوشته نشد؛ GitLab و Runner واقعی داخل همان کلاستر بالا آمدند، مخزن واقعی push شد و pipeline از build تا publish اجرا شد. مسیر داده از یک Job ترافیک ۱۰۰ هزار درخواستی، Nginx و سه backend شروع شد، با HDFS و پنج Streaming Job ادامه پیدا کرد و به ۱۳ خروجی تأیید شده و dashboard قابل مشاهده رسید.

خطاهای اصلی پروژه بیشتر از جنس اتصال اجزا بودند: DNS بین namespace host و cluster، کلیدهای پایدار، GitLab، مسیر دقیق، PVC نام متغیر، Mode Safe environment، HDFS، مصرف حافظه validator و permission کاربر. non-root. هر مورد پس از مشاهده در اجرای واقعی به کد یا تست رگرسیون تبدیل شد. در نتیجه نسخه تحویلی صرفاً یک نمونه نمایشی نیست و مسیر نصب و اجرای آن از روی اسکریپت‌ها قابل تکرار است.